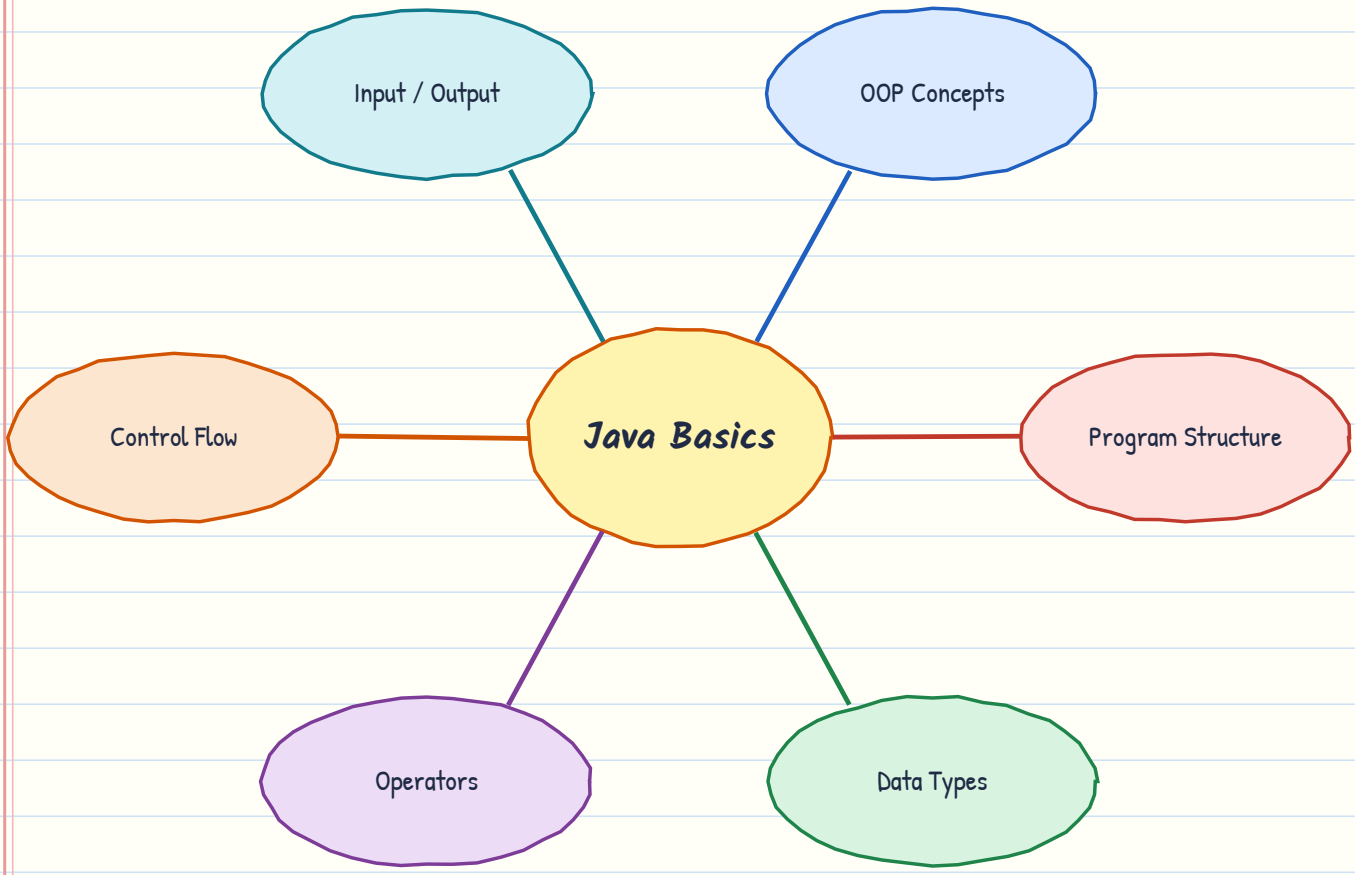
UNIT 1

OOP Basics, Data Types, Operators, Control

Program structure | Variables | Operators | Decisions & Loops



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

1. What is Object Oriented Programming?

OOP is a way of writing programs where we model real-world things as **objects**. Each object keeps its own ***data*** (state) and its own ***actions*** (behaviour) together.

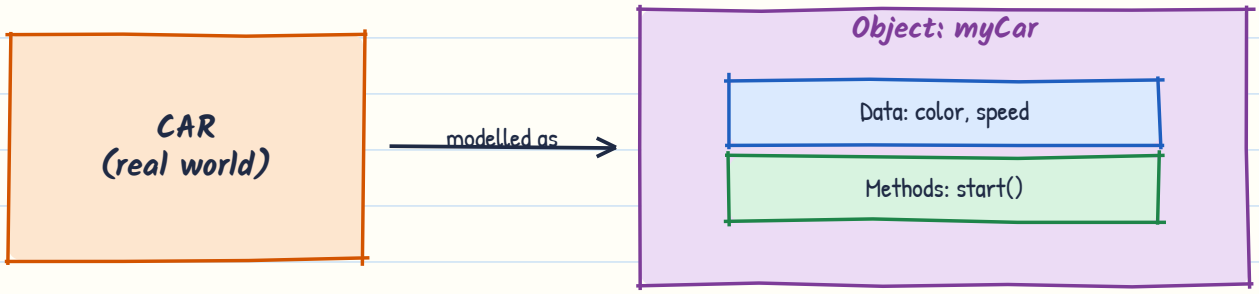


Fig 1: A real-world object -> a software object

Basic terms

Term	Meaning	Example
Class	Blueprint / template of objects	class Car
Object	Real instance made from a class	Car myCar = new Car();
Method	A function inside a class (behaviour)	start()
Message passing	Objects talk by calling methods	myCar.start()

The 4 Principles (pillars) of OOP

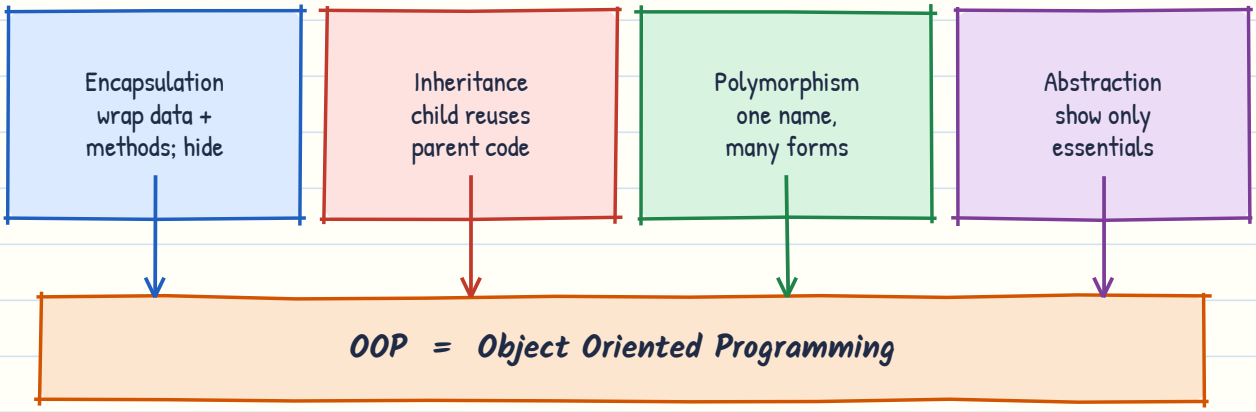


Fig 2: Four pillars of OOP

- ***Encapsulation*** - binding data and code in one unit (class); private data = safe data.
- ***Inheritance*** - a new class acquires properties of an existing class (code reuse).
- ***Polymorphism*** - same method name behaves differently (overloading / overriding).
- ***Abstraction*** - hide the "how", show only the "what" (abstract class, interface).

Memory trick

Easy memory trick: ATM = Abstraction hides the machinery, Encapsulation is the machine body, Polymorphism: "withdraw" works for savings or current, Inheritance: SavingsAccount IS-A Account.

2. Program Structure in Java

Every Java program is built from the parts shown below. Java source is saved as `ClassName.java`, compiled to ***bytecode*** (`.class`) and run on the ***JVM***.

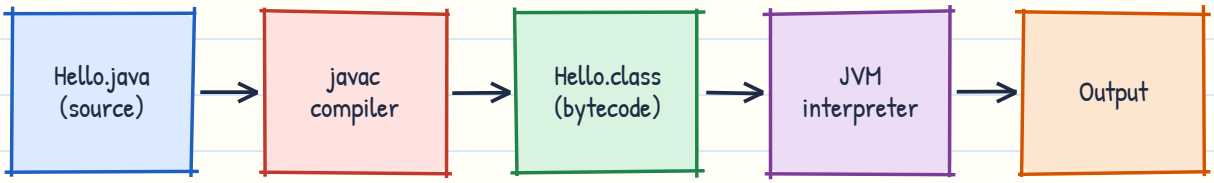


Fig 3: How a Java program runs (Write Once, Run Anywhere)

Structure of a simple program

Hello.java

```
import java.util.*;                  // 1. package (import) section
public class Hello {                 // 2. class declaration
    public static void main(String[] args) {   // 3. main method
        System.out.println("Hello, Java!");    // 4. statement
    }
}
```

```
> OUTPUT
Hello, Java!
```

- `public` - accessible to JVM from anywhere
- `static` - JVM calls main without creating an object
- `void` - main returns nothing
- `String[] args` - command line arguments array

Compile and run

```
javac Hello.java      // creates Hello.class
java Hello            // runs the program (no .class!)
```

Rule

The file name must match the public class name (case sensitive). Hello.java -> public class Hello.

Elements (Tokens) of a Java program

A *token* is the smallest meaningful unit the compiler reads. Java has five kinds:

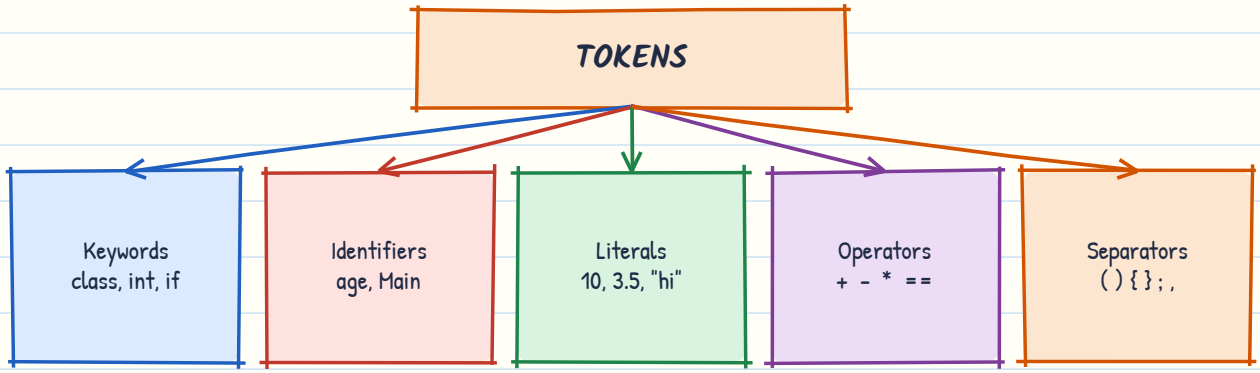


Fig 4: Java tokens

- *Identifier rules:* letters, digits, _, $; cannot start with a digit; no keywords; case-sensitive.
  - valid: `total`, `_x`, `$cost` | invalid: `2sum`, `my-var`, `class`
- Java has 50+ *reserved keywords* (they have fixed meaning).

Java statements

Type	Description	Example
Declaration	creates a variable	int a;
Expression	calculation / assignment	a = 5 + 2;
Control	changes flow of execution	if, for, while
Empty / Block	nothing / group inside { }	; or { ... }

Command Line Arguments

Values typed after the class name when running are stored as Strings in `args[]`.

Add.java *run: java Add 10 20*

```java
public class Add {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);   // text -> int
        int b = Integer.parseInt(args[1]);
        System.out.println("Sum = " + (a + b));
    }
}
```

```
> OUTPUT
Sum = 30
```

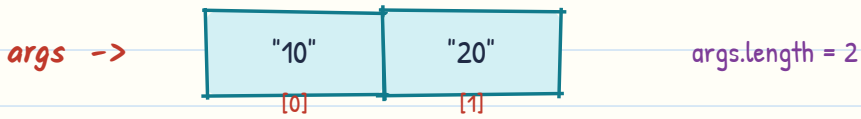


Fig 5: args[] after java Add 10 20

User input using Scanner

Input.java

```java
import java.util.Scanner;
public class Input {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        System.out.println(name + " is " + age);
        sc.close();
    }
}
```

Method	Reads	Method	Reads
nextInt()	int	nextDouble()	double
next()	one word	nextLine()	whole line
nextBoolean()	boolean	nextLong()	long

Escape sequences & comments

Escape	Meaning	Escape	Meaning
\n	new line	\t	tab space
\"	double quote	\\	backslash
\'	single quote	\r	carriage return

```
// single line comment
/* multi
   line comment */
/** documentation comment (javadoc) */
```

Good style

Programming style: meaningful names, camelCase for variables/methods (totalMarks), PascalCase for classes (StudentRecord), UPPER_CASE for constants, indent with 4 spaces, one statement per line.

3. Data Types, Variables & Operators

Java is ***strongly typed***: every variable must have a declared type. Types are divided into two families.

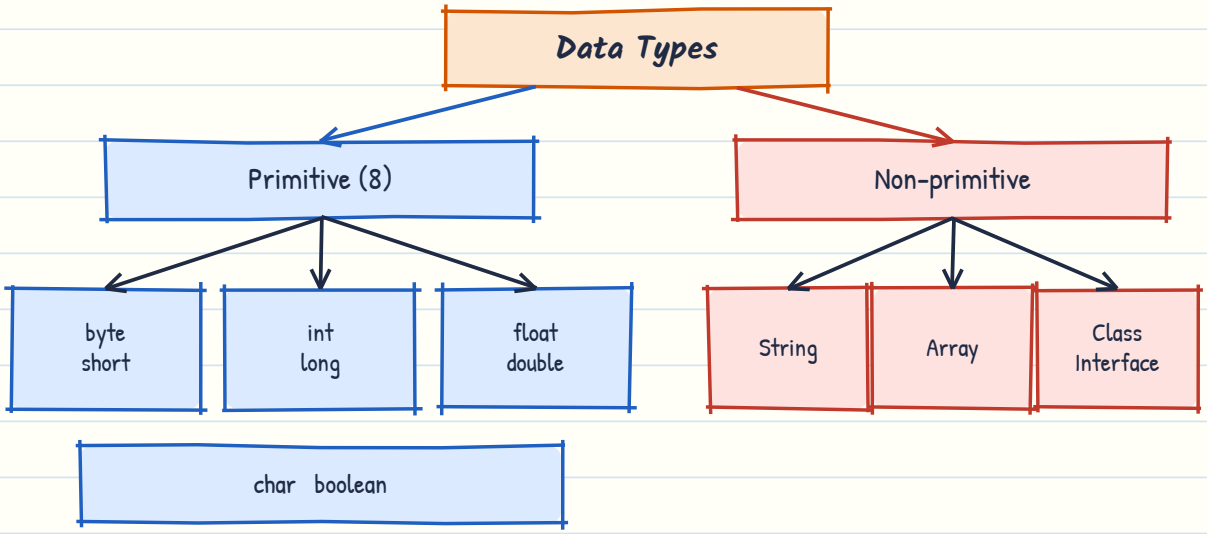


Fig 6: Classification of Java data types

Type	Size	Range / Use	Default
byte	1 byte	-128 to 127	0
short	2 bytes	-32,768 to 32,767	0
int	4 bytes	about -2.1 to +2.1 billion	0
long	8 bytes	very large whole numbers (123L)	0L

float	4 bytes	decimal, 6-7 digits (3.5f)	0.0f
double	8 bytes	decimal, 15 digits (default)	0.0
char	2 bytes	single Unicode char ('A')	\u0000
boolean	1 bit*	true / false	false

Declaring variables

```
int age;                    // declaration
age = 20;                   // assignment
double price = 99.5;        // declaration + initialization
int a = 5, b = 10, c;       // many at once
final double PI = 3.14159;   // constant (cannot change)
```

Type casting

Converting one data type into another. ***Widening*** is automatic and safe; ***narrowing*** needs a manual cast and may lose data.

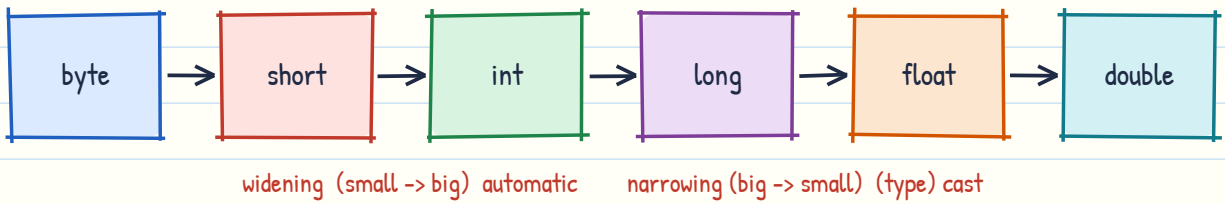


Fig 7: Widening (auto) -> and Narrowing (manual) <-

```
int i = 100;
double d = i;             // widening: 100.0
double pi = 9.78;
int x = (int) pi;         // narrowing: 9  (decimal lost)
char ch = (char) 66;      // 'B'
```

Scope of a variable

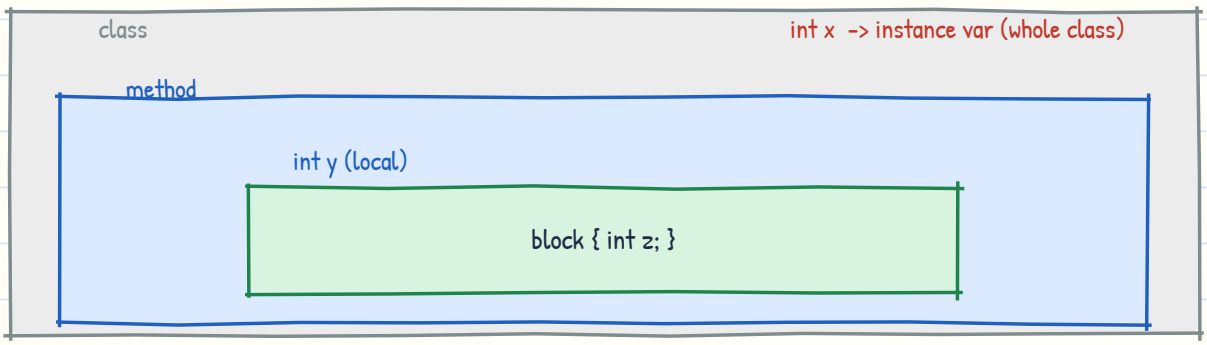


Fig 8: Variable scope - where a variable is alive

- ***Instance variable*** - declared in class, one copy per object.

- ***Local variable*** – declared inside method/block; dies when block ends; no default value.
- ***Static variable*** – one shared copy for the whole class.

Literals, constants & printf()

Literal constant = fixed value written in code: `10`, `3.14`, `'A'`, `"Java"`, `true`. ***Symbolic constant*** = named constant using `final`: `final int MAX = 100;`

```
System.out.printf("Name: %s, Age: %d%n", "Ravi", 21);
System.out.printf("Price: %.2f%n", 99.5);
System.out.printf("[%5d] [%-5d]%n", 42, 42);
```

```
> OUTPUT
Name: Ravi, Age: 21
Price: 99.50
[   42] [42   ]
```

Format	Prints	Format	Prints
%d	int	%f	decimal
%s	String	%c	char
%.2f	2 decimals	%n	new line
%b	boolean	%x	hex

static and final

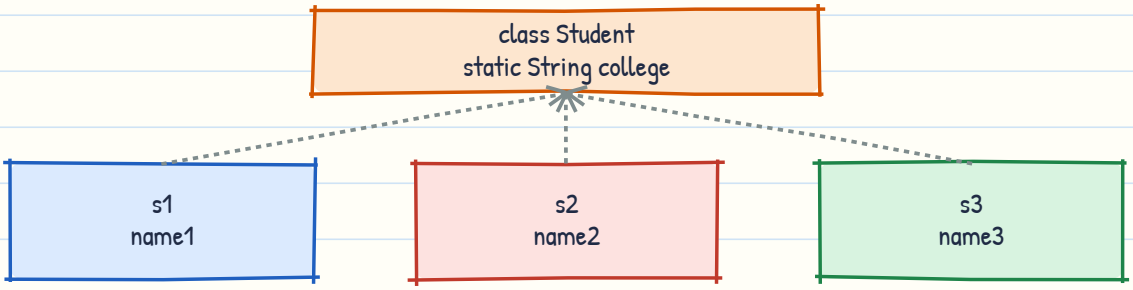


Fig 9: static variable is shared by all objects

- `static` variable/method belongs to the ***class***; call as `ClassName.method()`; static methods cannot use `this` or non-static members directly.
- `final` variable = constant; `final` method = cannot be overridden; `final` class = cannot be inherited.

Operators

Category	Operators	Example
Arithmetic	+ - * / %	a % b -> remainder
Assignment	= += -= *= /=	x += 5 (x = x+5)
Increment / Decrement	++ --	i++ ++i
Relational	!= > < >=	a > b

Logical	&& || !	(a>1) && (b<5)
Bitwise	& | ^ ~ << >> >>>	5 & 3 = 1
Ternary	cond ? x : y	max = a>b ? a : b

j = i++
use first (j=5)
then i becomes 6

j = ++i
increase first (i=6)
then j = 6

Fig 10: Pre vs post increment, int i = 5

```
int a = 10, b = 3;
System.out.println(a / b);      // 3     (int division)
System.out.println(a % b);      // 1
System.out.println(a > b && b > 0);   // true
System.out.println(5 & 3);      // 1   (101 & 011 = 001)
System.out.println(5 | 3);      // 7
System.out.println(5 ^ 3);      // 6
int max = (a > b) ? a : b;      // 10
```

Precedence & associativity (high -> low)

Level	Operators	Associativity
1	() [] .	left to right
2	++ -- ! ~ (unary)	right to left
3	* / %	left to right
4	+ -	left to right
5	<< >>	left to right
6	< <= > >=	left to right
7		left to right
8	& ^ |	left to right
9	&& then ||	left to right
10	?:	right to left
11	= += -=	right to left

Tip

BODMAS-like: brackets first, then * / %, then + -. When confused, just add ( ) !

4. Control Statements

Control statements decide **which** code runs and **how many times**. Two families: selection (decisions) and iteration (loops), plus jump statements.

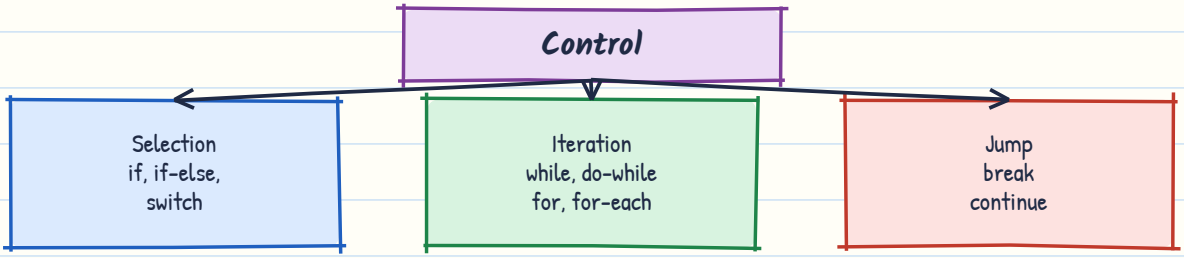


Fig 11: Control statements

if, if-else, nested if

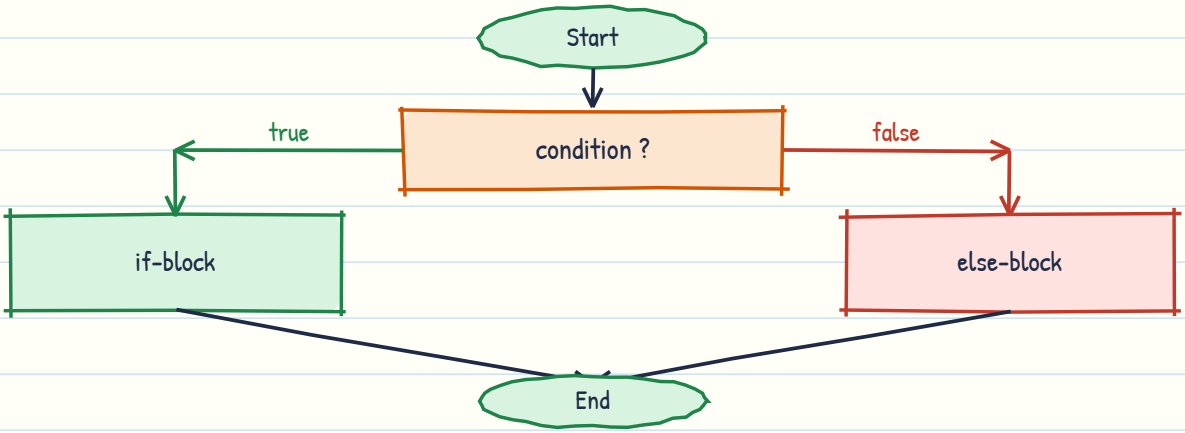


Fig 12: Flowchart of if-else

```java
int marks = 72;
if (marks >= 90)       System.out.println("Grade A");
else if (marks >= 60)  System.out.println("Grade B");   // runs
else                   System.out.println("Fail");
```

```
> OUTPUT
Grade B
```

Nested if: an `if` inside another `if` - inner one is tested only when the outer is true.

```java
if (age >= 18) {
    if (hasId)  System.out.println("Allowed");
    else        System.out.println("Bring ID");
}
```

switch statement

```
int day = 3;
switch (day) {
    case 1:  System.out.println("Mon"); break;
    case 2:  System.out.println("Tue"); break;
    case 3:  System.out.println("Wed"); break;    // runs
    default: System.out.println("Other");
}
```

Common mistake

Forget `break` and execution FALLS THROUGH into the next case! switch works with int, char, String, enum (not long/float/double).

Iteration statements

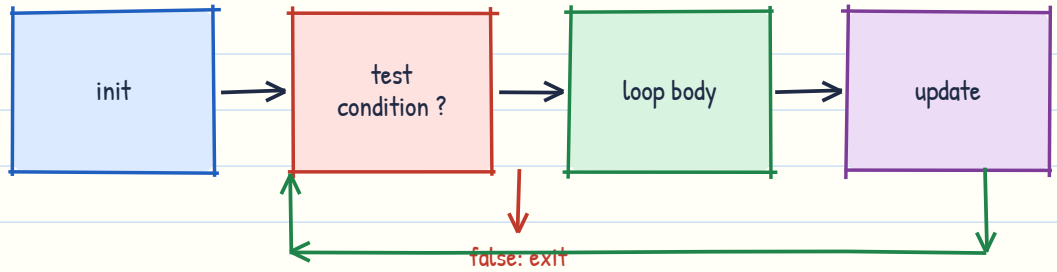


Fig 13: Loop flow (entry-controlled: while / for)

Loop	Type	Use when
while	entry-controlled	number of repeats unknown
do-while	exit-controlled	body must run at least once
for	entry-controlled	number of repeats known
for-each	collection loop	visit every array element

```
int i = 1;
while (i <= 3) { System.out.print(i + " "); i++; }     // 1 2 3
int j = 10;
do { System.out.println("runs once"); } while (j < 5);
for (int k = 1; k <= 5; k++) System.out.print(k + " ");  // 1 2 3 4 5
int[] a = {10, 20, 30};
for (int v : a) System.out.print(v + " ");           // 10 20 30
```

Nested for loop - pattern

```java
for (int i = 1; i <= 4; i++) {          // rows
    for (int j = 1; j <= i; j++)       // columns
        System.out.print("* ");
    System.out.println();
}
```

```
> OUTPUT
*
* *
* * *
* * * *
```

break and continue

break;	continue;
exit the loop completely	skip rest of body, next iteration

Fig 14: break leaves the loop, continue skips one round

```java
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;      // skips 3
    if (i == 5) break;         // stops at 5
    System.out.print(i + " ");
}
```

```
> OUTPUT
1 2 4
```

Quick Revision & Exam Questions

Topic	One-line memory
OOP pillars	Encapsulation, Inheritance, Polymorphism, Abstraction
Execution	.java -> javac -> .class -> JVM
Tokens	Keyword, Identifier, Literal, Operator, Separator
Casting	Widening auto, narrowing manual (type)
static	belongs to class, one shared copy
final	variable=constant, method=no override, class=no inherit
switch	needs break, else fall-through
Loops	while/for check first, do-while checks last

Likely exam questions

- Explain the basic concepts and principles of OOP with examples.
- Describe the structure of a Java program; explain compile and run steps.
- Differentiate primitive and non-primitive data types; explain type casting.
- Explain static and final with examples.
- Explain operator precedence; write a note on bitwise operators.
- Differentiate while, do-while and for; explain break and continue.
- Write a program to read a number with Scanner and print its factorial.